UNIVERSITY OF SOUTHERN CALIFORNIA- FALL 2024

CSCI401 CAPSTONE PROJECT
TEAM 11 FINAL PRESENTATION

LAFD AI HELP DESK CHATBOT

OVERVIEW

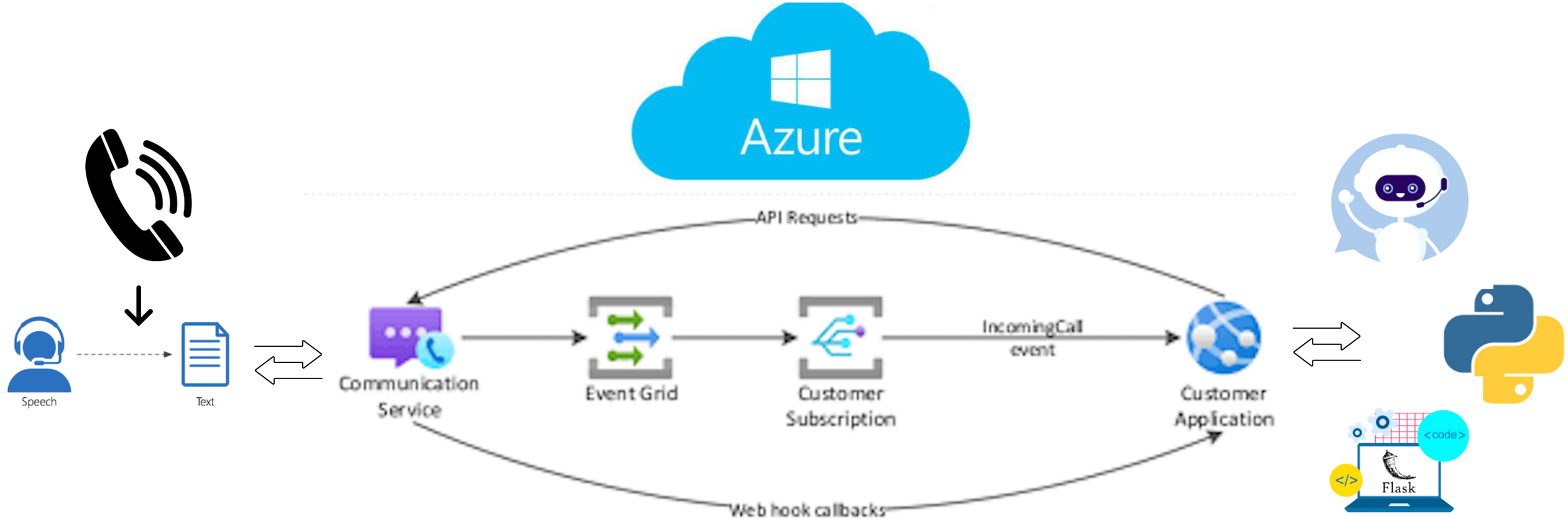
OVERVIEW

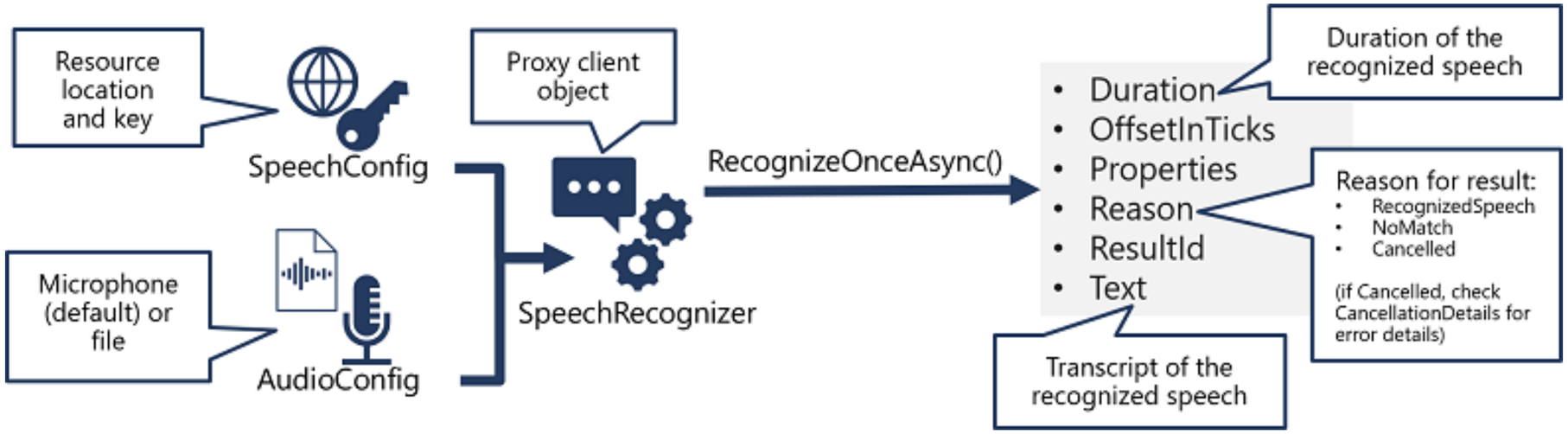
- This project, developed during the Fall 2024 semester of the USC CSCI401 Capstone course, focuses on creating an AI-powered chatbot solution for the Los Angeles Fire Department (LAFD) Help Desk. Traditionally, incoming service calls have been handled either by on-duty agents or through a basic web-based form, resulting in bottlenecks and increased pressure during high-volume periods. To address these challenges, this application leverages advanced, cloud-based technologies—primarily within the Microsoft Azure ecosystem—to provide a more efficient and scalable approach to managing technical service requests.

MAJOR TECHNOLOGIES UTILIZED

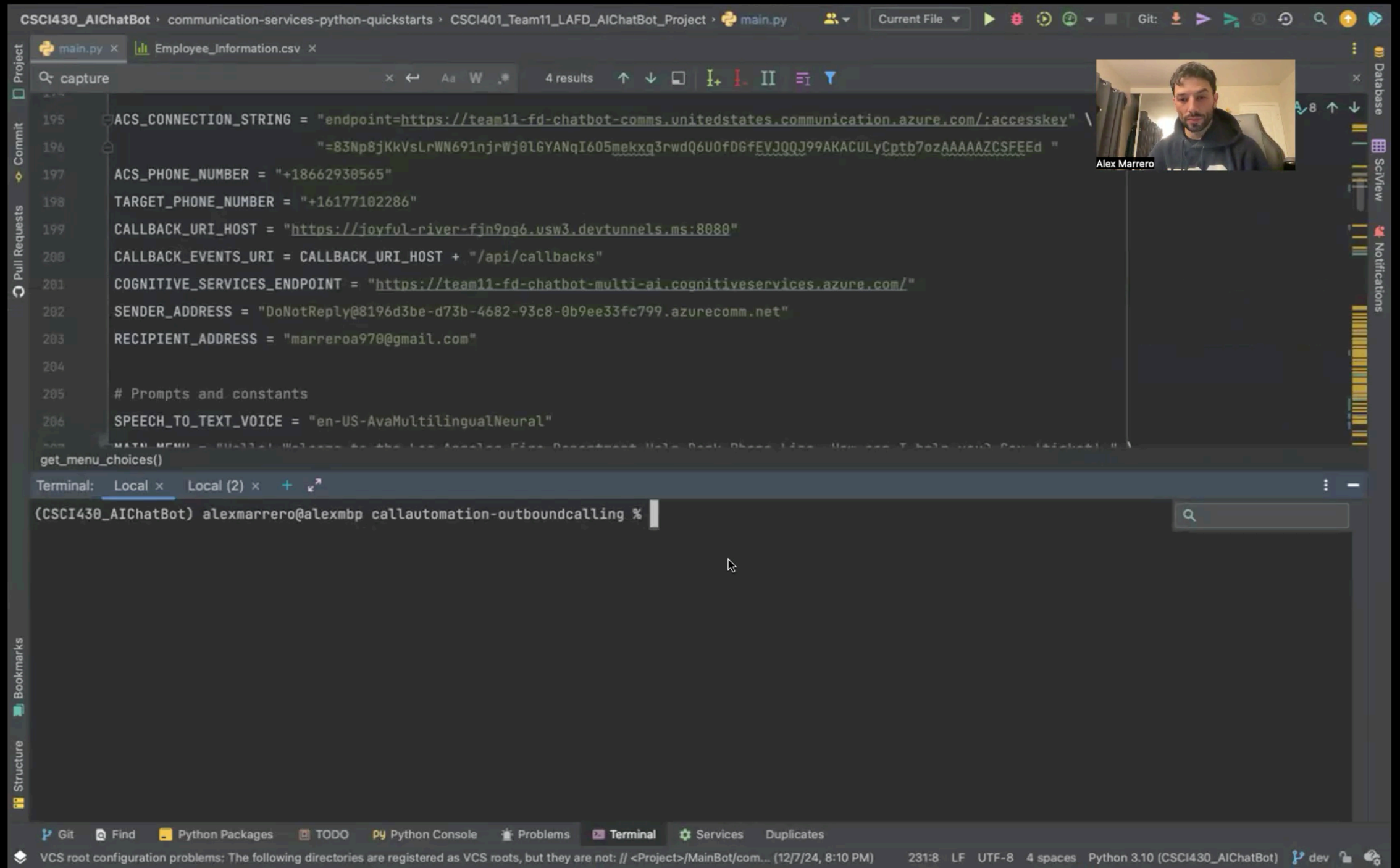
MAJOR TECHNOLOGIES USED

- Azure Cognitive Services (Speech to Text / Text to Speech
- Azure Call Automation API
- Python / PyCharm IDE
- Azure Communications Resources
- Microsoft AI Services
- Flask Python Web App Framework


Azure
API Requests
Speech
Text
Communication Service
Event Grid
Customer Subscription
IncomingCall event
Customer Application
Web hook callbacks
<code>
Flask


Resource location and key
SpeechConfig
Proxy client object
RecognizeOnceAsync()
Duration
OffsetInTicks
Properties
Reason
ResultId
Text
Duration of the recognized speech
Reason for result:
RecognizedSpeech
NoMatch
Cancelled
(if Cancelled, check CancellationDetails for error details)
Microphone (default) or file
AudioConfig
SpeechRecognizer
Transcript of the recognized speech

APPLICATION DEMONSTRATION


CSCI430_AIChatBot › communication-services-python-quickstarts › CSCI401_Team11_LAFD_AIChatBot_Project › main.py
Current File
Git:
main.py
Employee_Information.csv
capture
4 results
ACS_CONNECTION_STRING = "endpoint=https://team11-fd-chatbot-comms.unitedstates.communication.azure.com/;accesskey" \
"=83Np8jKkVsLrWN691njrWj0lGYANqI6O5mekxq3rwdQ6UOfDGfEVJQQJ99AKACULyCptb7ozAAAAAZCSFEEd "
ACS_PHONE_NUMBER = "+18662930565"
TARGET_PHONE_NUMBER = "+16177102286"
CALLBACK_URI_HOST = "https://joyful-river-fjn9pg6.usw3.devtunnels.ms:8080"
CALLBACK_EVENTS_URI = CALLBACK_URI_HOST + "/api/callbacks"
COGNITIVE_SERVICES_ENDPOINT = "https://team11-fd-chatbot-multi-ai.cognitiveservices.azure.com/"
SENDER_ADDRESS = "DoNotReply@8196d3be-d73b-4682-93c8-0b9ee33fc799.azurecomm.net"
RECIPIENT_ADDRESS = "marreroa970@gmail.com"
# Prompts and constants
SPEECH_TO_TEXT_VOICE = "en-US-AvaMultilingualNeural"
Alex Marrero
get_menu_choices()
Terminal:
Local
Local (2)
(CSCI430_AIChatBot) alexmarrero@alexmbp callautomation-outboundcalling %
Git
Find
Python Packages
TODO
Python Console
Problems
Terminal
Services
Duplicates
VCS root configuration problems: The following directories are registered as VCS roots, but they are not: // <Project>/MainBot/com... (12/7/24, 8:10 PM)
231:8
LF
UTF-8
4 spaces
Python 3.10 (CSCI430_AIChatBot)
dev

QUICK CODE EXPLANATION

CODE EXPLANATION

- Initialize Data Models: Begin by defining the Employee and Ticket classes, ensuring a structured representation of employee data and service ticket information
- Load and Prepare Data: Read the Employee_Information.csv file into a dictionary of Employee objects, providing a quick reference for caller identification and information retrieval
- Global Configuration: Set up global variables, including credential strings, Azure service endpoints, and other constants required for integrating speech services, call automation, and event handling.

CUSTOM OBJECTS

```python
class Employee:
    """Represents an employee with associated contact and identification details.

    Attributes:
        employee_id (str): The unique employee ID.
        first_name (str): The employee's first name.
        last_name (str): The employee's last name.
        display_name (str): A formatted display name (usually first + last name).
        telephone_number (str): The employee's phone number.
        email_address (str): The employee's email address.
    """

    Alex Marrero
    def __init__(self, employee_id, first_name, last_name, display_name, telephone_number, email_address):
        """Initialize an Employee instance.

        Args:
            employee_id (str): Unique identifier for the employee.
            first_name (str): Employee's first name.
            last_name (str): Employee's last name.
            display_name (str): Employee's display name.
            telephone_number (str): Employee's telephone number.
            email_address (str): Employee's email address.
        """
        self.employee_id = employee_id
        self.first_name = first_name
        self.last_name = last_name
```

```python
class Ticket:
    """Represents a help desk ticket that captures user details and the issue.

    Attributes:
        name (str): Name of the employee who filed the ticket.
        id_num (str): Employee ID number.
        contact_method (str): Preferred contact method (e.g., 'phone', 'email').
        phone_number (str): The phone number to contact.
        email (str): The email address to contact.
        work_mode (str): The employee's work mode (e.g., 'office', 'telework').
        work_address (str): The physical location of the issue.
        urgency (str): Urgency level of the issue ('low', 'medium', 'high').
        issue_description (str): A description of the reported issue.
    """

    Alex Marrero
    def __init__(self, name="", id_num="", contact_method="", phone_number="", email="", work_mode="", work_address="",
                 urgency="", issue_description=""):
        """Initialize a Ticket instance...."""
        self.name = name
        self.id_num = id_num
        self.contact_method = contact_method
        self.phone_number = phone_number
        self.email = email
        self.work_mode = work_mode
        self.work_address = work_address
        self.urgency = urgency
```

Find | Python Packages | TODO | Python Console | Problems | Terminal | Services | Duplicates

nfiguration problems: The following directories are registered as VCS roots, but they are not: // <Project>/MainBot/co... (12/7/24, 8:10 PM) 44:8 LF UTF-8 4 space

Alex Marrero

```python
class RetryObject:
    """Keeps track of retries for speech/choice recognition attempts during IVR interactions...."""

    Alex Marrero
    def __int__(self, context="", choices=[], mode="", counter=0):
        """Initialize a RetryObject instance.

        Args:
            context (str): The current recognition context.
            choices (list): A list of choices to present to the user again if retries occur.
            mode (str): The mode of recognition ('speech', 'choices', 'speech_or_dtmf').
            counter (int): Retry attempt counter.
        """
        self.context = context
        self.choices = choices
        self.mode = mode
        self.counter = counter
```

```python
# Employee data loading
employee_dict = {}
with open('EmployeeInfo/Employee_Information.csv', mode='r') as file:
    csv_reader = csv.DictReader(file)
    for row in csv_reader:
        employee = Employee(
            employee_id=row["ID"],
            first_name=row["FirstName"],
            last_name=row["LastName"],
            display_name=row["DisplayName"],
            telephone_number=row["TelephoneNumber"],
            email_address=row["EmailAddress"],
        )
        employee_dict[employee.employee_id] = employee

# Global variables for ticket processing and call handling
employee_ids = list(employee_dict.keys())
current_employee: Employee = None
new_phone_number = None
current_ticket = Ticket()
```

```
ACS_CONNECTION_STRING = "ENTER ACS CONNECTION STRING HERE"
ACS_PHONE_NUMBER = "ENTER ACS PHONE NUMBER HERE"
CALLER_PHONE_NUMBER = ""
CALLBACK_URI_HOST = "ENTER CALLBACK URL HERE"
CALLBACK_EVENTS_URI = CALLBACK_URI_HOST + "/api/callbacks"
COGNITIVE_SERVICES_ENDPOINT = "ENTER AZURE AI MULTI SERVICE RESOURCE ENDPOINT HERE"
SENDER_ADDRESS = "ENTER AZURE EMAIL RESOURCE MAIL-FROM ADDRESS HERE"
RECIPIENT_ADDRESS = "www.tech.footprints@fire.lacounty.gov"

# Prompts and constants
SPEECH_TO_TEXT_VOICE = "en-US-AvaMultilingualNeural"
MAIN_MENU = "Hello! Welcome to the Los Angeles Fire Department Help Desk Phone Line. ..."
CUSTOMER_QUERY_TIMEOUT = "I’m sorry I didn’t receive a response, please try again."
NO_RESPONSE = "I didn't receive an input ..."
INVALID_AUDIO = "I’m sorry, I didn’t understand your response, please try again."
CONFIRM_CHOICE_LABEL = "Confirm"
CANCEL_CHOICE_LABEL = "No"
TICKET_CHOICE_LABEL = "Ticket"
MCU_CHOICE_LABEL = "MCU"
RETRY_CONTEXT = "retry"
GOODBYE = "Goodbye for now!"
FATAL_ERROR_MESSAGE = "I apologize, but there was a systematic error in the call. Please call again."

call_automation_client = CallAutomationClient.from_connection_string(ACS_CONNECTION_STRING)
```

CODE EXPLANATION

- Azure Integration: Initialize the Azure CallAutomationClient and other AI resources to support speech-to-text, text-to-speech, and event-driven call flows.
- Inbound Call Entry Point: The /inboundCall route receives initial notifications from Azure about incoming calls, handles subscription validations, and answers incoming calls.
- Event-Driven Logic: The /api/callbacks route processes a stream of call-related events (e.g., call connected, speech recognized, digits pressed), each event triggering specific logic paths to guide the caller through the menu and data collection steps.
- Modular Prompts and Decision Points: Individual functions manage distinct parts of the call flow—collecting employee ID, verifying contact info, determining urgency, and capturing issue descriptions—making the application highly modular and adaptable.

```python
def get_media_recognize_choice_options(call_connection_client: CallConnectionClient, text_to_play: str,
                                       target_participant: str, choices: list, context: str):
    """Prompt the user with a set of menu choices (DTMF or speech).

    Args:
        call_connection_client (CallConnectionClient): The call connection client to send recognition requests.
        text_to_play (str): The prompt text to play to the user.
        target_participant (str): The participant's phone number or identifier.
        choices (list): A list of RecognitionChoice options.
        context (str): The operation context for this recognition attempt.
    """
    retry_object.context = context
    retry_object.mode = "choices"
    retry_object.choices = choices
    app.logger.info("Bot Dialogue: '%s'", text_to_play)
    play_source = TextSource(text=text_to_play, voice_name=SPEECH_TO_TEXT_VOICE)
    call_connection_client.start_recognizing_media(
        input_type=RecognizeInputType.CHOICES,
        target_participant=target_participant,
        choices=choices,
        play_prompt=play_source,
        interrupt_prompt=True,
        initial_silence_timeout=30,
        end_silence_timeout=2,
        operation_context=context
    )

n_choices()
```

Alex Marrero

```python
def get_media_recognize_speech_or_dtmf_input(call_connection_client: CallConnectionClient, text_to_play: str,
                                             target_participant: str, context: str):
    """Prompt the user for speech or DTMF input..."""
    retry_object.context = context
    retry_object.mode = "speech_or_dtmf"
    play_source = TextSource(text=text_to_play, voice_name=SPEECH_TO_TEXT_VOICE)

    # Determine max tones depending on context
    if context == "provide_eid":
        MAX_TONES = 6
    else:
        MAX_TONES = 10

    call_connection_client.start_recognizing_media(
        dtmf_max_tones_to_collect=MAX_TONES,
        input_type=RecognizeInputType.SPEECH_OR_DTMF,
        target_participant=target_participant,
        end_silence_timeout=2,
        play_prompt=play_source,
        initial_silence_timeout=30,
        interrupt_prompt=False,
        operation_context=context
    )
    app.logger.info("Start recognizing speech or DTMF input")
```

Alex Marrero

```python
def get_media_recognize_speech_input(call_connection_client: CallConnectionClient, text_to_play: str,
                                     target_participant: str, context: str):
    """Prompt the user for speech input only.

    Args:
        call_connection_client (CallConnectionClient): The call connection client.
        text_to_play (str): The prompt to the user.
        target_participant (str): The participant's identifier.
        context (str): The recognition context.
    """
    retry_object.context = context
    retry_object.mode = "speech"
    play_source = TextSource(text=text_to_play, voice_name=SPEECH_TO_TEXT_VOICE)
    call_connection_client.start_recognizing_media(
        input_type=RecognizeInputType.SPEECH,
        target_participant=target_participant,
        play_prompt=play_source,
        interrupt_prompt=False,
        end_silence_timeout=2,
        initial_silence_timeout=30,
        operation_context=context
    )
    app.logger.info("Start recognizing speech input")
```

Alex Marrero

```python
@app.route('/inboundCall', methods=['POST'])
def inbound_call_handler():
    """Handle inbound calls from Azure Event Grid.

    This endpoint receives events indicating incoming calls (and validation events).
    On a SubscriptionValidationEvent, it returns the validation code.
    On an IncomingCall event, it answers the call and sets the caller as the target.

    Returns:
        A Flask redirect response after processing the event.
    """
    events = request.get_json()
    app.logger.info("Received Event: %s", events)

    for event in events:
        event_type = event.get('eventType')
        event_data = event.get('data', {})

        if event_type == "Microsoft.EventGrid.SubscriptionValidationEvent":
            validation_code = event_data.get('validationCode')
            app.logger.info("Validation Code: %s", validation_code)
            return jsonify({"validationResponse": validation_code}), 200

        elif event_type == "Microsoft.Communication.IncomingCall":
            incoming_call_context = event_data.get('incomingCallContext')
            caller_id = event_data.get('from', {}).get('phoneNumber', 'Unknown').get('value', 'Unknown')
```

```python
@app.route('/api/callbacks', methods=['POST'])
def callback_events_handler():
    """Handle callback events from Azure Communication Services.

    This endpoint processes various events (Connected, RecognizeCompleted, RecognizeFailed, etc.)
    and orchestrates the IVR logic, prompting the user for inputs, confirming details,
    and logging the ticket.

    Returns:
        flask.Response: A 200 OK response after handling the events.
    """
    for event_dict in request.json:
        # Parsing callback events
        event = CloudEvent.from_dict(event_dict)
        call_connection_id = event.data['callConnectionId']
        app.logger.info("%s event received for call connection id: %s", event.type, call_connection_id)
        call_connection_client = call_automation_client.get_call_connection(call_connection_id)
        target_participant = PhoneNumberIdentifier(CALLER_PHONE_NUMBER)
        if event.type == "Microsoft.Communication.CallConnected":
            reset_ticket()
            app.logger.info("Starting recognize")
            get_media_recognize_choice_options(
                call_connection_client=call_connection_client,
                text_to_play=MAIN_MENU,
                target_participant=target_participant,
```

```
# SPEECH INPUT RECOGNIZED
elif event.data['recognitionType'] == "speech":
    text = event.data['speechResult']['speech'];
    app.logger.info("Recognition completed, text=%s, context=%s", text,
                    event.data.get('operationContext'));

    if event.data['operationContext'] == "provide_eid":
        text = text.lower()
        no_spaces_text = text.replace(" ", "")
        final_id_num = no_spaces_text.replace(".", "")
        temp_employee = get_employee_by_id(final_id_num)

        if isinstance(temp_employee, Employee):
            modify_current_employee(temp_employee)
            text_to_play = 'Great. Give me a moment while I look up your information... Is your name ' + \
                           current_employee.first_name + '?'
            get_media_recognize_choice_options(
                call_connection_client=call_connection_client,
                text_to_play=text_to_play,
                target_participant=target_participant,
                choices=get_confirm_choices(), context="confirm_name")

        else:
            text_to_play = "I couldn't find an employee under that I.D. number. Let's try one more time. " \
                           "Can you please say your employee ID number? "
            get_media_recognize_speech_or_dtmf_input(
```

- Ticket Completion and Sending
- Once all necessary information is gathered, the code uses send_email() to transmit the finalized ticket details to the designated Microsoft Footprints email address, completing the end-to-end help desk workflow.

```python
def send_email(ticket=None):
    """Send an email with the current ticket details...."""
    global current_ticket
    if ticket is None:
        ticket = current_ticket

    retry_object.reset()
    POLLER_WAIT_TIME = 5
    message = {
        "senderAddress": SENDER_ADDRESS,
        "recipients": {
            "to": [{"address": RECIPIENT_ADDRESS}],
        },
        "content": {
            "subject": "Help desk ticket",
            "plainText": str(ticket),
        }
    }

    try:
        client = EmailClient.from_connection_string(ACS_CONNECTION_STRING)
        poller = client.begin_send(message)
        time_elapsed = 0
        while not poller.done():
            app.logger.info("Email send poller status: " + poller.status())
            poller.wait(POLLER_WAIT_TIME)
```

LOCAL AND GLOBAL IMPACT

LOCAL AND GLOBAL IMPACT

- Streamlines the LAFD Help Desk by automating routine requests, reducing agent workload, and improving response times.
- Enhances internal operations, leading to quicker issue resolutions and higher employee satisfaction.
- Serves as a global example for organizations seeking AI-driven, scalable customer support solutions.
- Aligns with both organizational goals and broader market trends focusing on efficiency, user-centricity, and adaptability.

CONTINUED PROFESSIONAL DEVELOPMENT & TOOLS / SOFTWARE ENGINEERING PRACTICES

CONTINUING PROFESSIONAL DEVELOPMENT

- Mastery of cloud platforms (e.g., Azure services) for scalable, on-demand computing resources.
- Implementation of AI technologies for speech recognition and text-to-speech capabilities.
- Proficiency in Python Flask web development beyond basic server-side programming.
- Insight into telecommunication and call automation concepts unfamiliar to standard coursework.
- Comfort working with external APIs, Azure SDKs, and documentation as primary learning resources.

TOOLS, TECHNOLOGIES, AND SOFTWARE ENGINEERING PRACTICES

FAMILIAR TOOLS

- GitHub for version control (CSCI 301)
- PyCharm as a development environment, and applying basic front-end web development and Python scripting techniques (CSCI 360/301)
- Basic object oriented programming with classes and encapsulated functions (CSCI103/104).
- Web application development (CSCI301)

TOOLS, TECHNOLOGIES, AND SOFTWARE ENGINEERING PRACTICES

NEW TOOLS AND PRACTICES

- Employed Azure Event Grid for asynchronous, event-driven handling of call states and responses.
- Integrated Python Flask applications to cloud APIs
- Integrated Azure Communication Services for telephony and call automation features.
- Experience with real-world stakeholders

ETHICAL AND NON-TECHNICAL CONSIDERATIONS

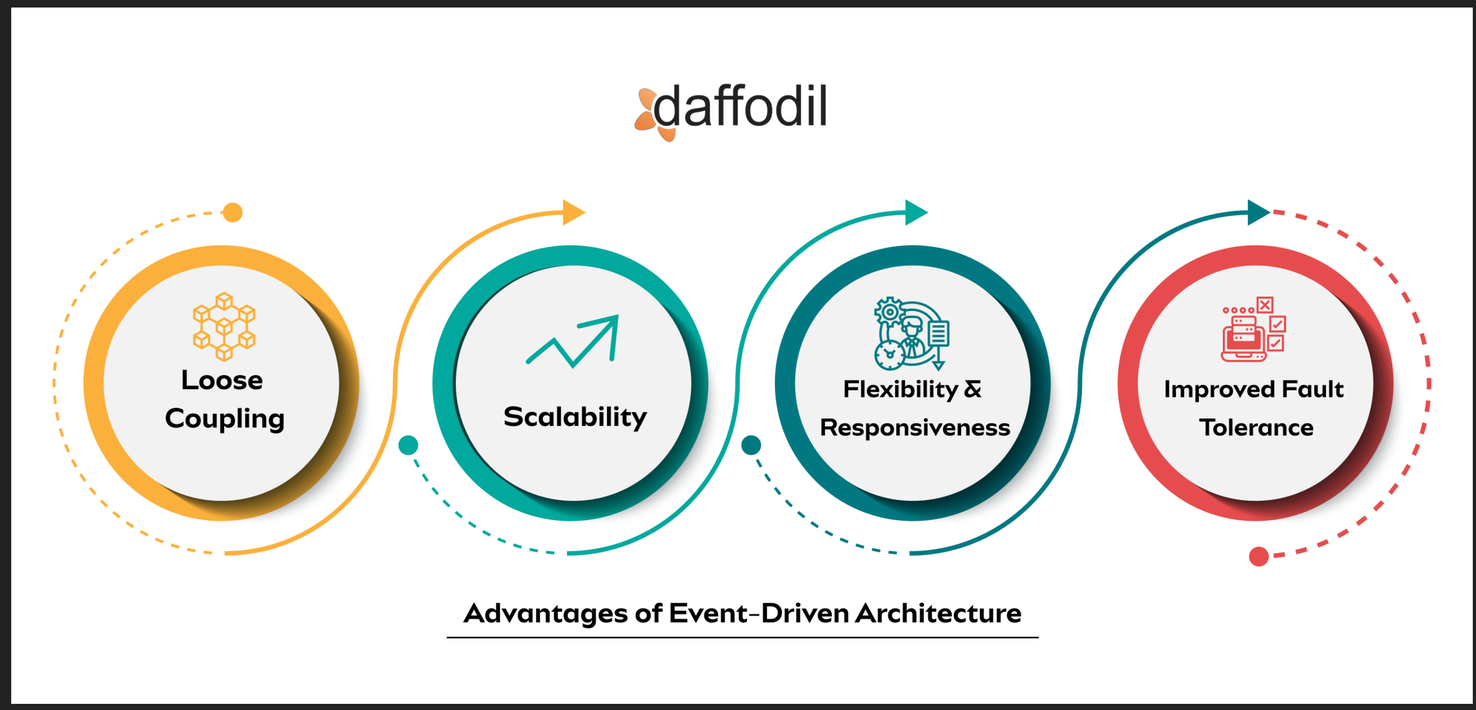
ETHICAL AND NON-TECHNICAL CONSIDERATIONS

- Minimal to no reference code required relying heavily on official documentation and internal trial and error.
- Dynamic and highly specialized integration tasks limited the utility of AI tools like ChatGPT.
- Confined network settings and familiar stakeholders reduced confidentiality and compliance concerns.
- Stakeholders, versed in IT complexities, set realistic expectations and were supportive of extended learning curves.
- Ethical and legal implications were minimal due to the internal, secure, and non-public nature of the system.

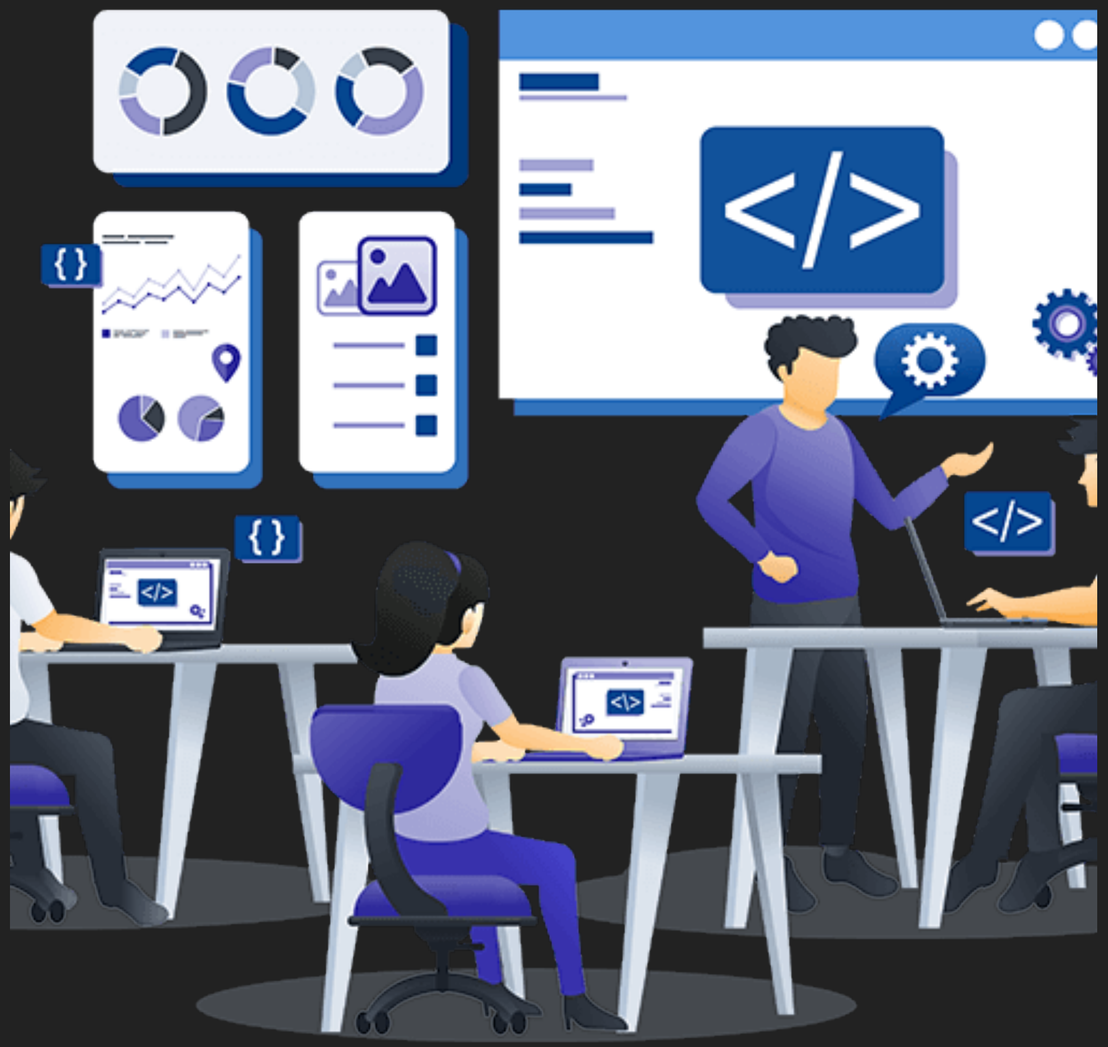
ARCHITECTURE AND DESIGN CHOICES

ARCHITECTURE AND DESIGN CHOICES

- Achieved a modular design where each event and component could be developed and tested independently.
- Reduced risk by isolating changes and updates to small, manageable units of logic.
- Ensured alignment with project requirements by improving flexibility, maintainability, and incremental problem-solving capabilities.
- Shifted from traditional Agile sprints to an Event-Driven Architecture due to high uncertainty and unfamiliar tools.
- Focused on individual event handling in the call flow before integrating all parts into a cohesive solution.



TEAM WORKFLOW AND COLLABORATION



TEAM WORKFLOW AND COLLABORATION

- Team members self-selected into roles that matched their strengths, from technical coding to stakeholder liaison work.
- Communication was largely remote, with video calls or in-person meetings reserved for critical, time-sensitive challenges.
- Roles became clearer as the project progressed, ensuring all aspects of development and coordination were covered.
- Flexibility in task assignments allowed the team to adapt to the project's non-deterministic nature and unfamiliar technologies.
- This hybrid, role-driven approach supported both technical innovation and effective stakeholder engagement.

FUTURE CONSIDERATIONS AND PERSONAL COMMENTS

FUTURE CONSIDERATIONS AND PERSONAL COMMENTS

- Implement automated data cleaning and validation scripts for the employee information file to prevent formatting errors and ensure consistent data quality.
- Establish direct routing configurations to seamlessly integrate the radio-controlled unit's on-premises telephony system with Azure services, enabling full call transfer capabilities.
- Develop a full-scale front-end interface and upgrade to a production-grade web server (e.g., Azure App Service) for improved scalability, reliability, and user experience. Alternatively, the subsequent groups can utilize a web server that functions alongside the LAFD infrastructure (if applicable)
- Integrate comprehensive logging and optional call recording features for quality assurance, compliance, and ongoing performance monitoring.
- Expand the application's language capabilities to support multilingual interactions, enhancing accessibility and inclusivity for a diverse user base.

FUTURE CONSIDERATIONS AND PERSONAL COMMENTS

- We would like to personally thank Tony, Bruce, Halim, and the entire LAFD Information Systems team for contributing to our capstone experience in such a positive way. They have helped us grow as professionals and as developers by providing a comfortable relationship and achievable goals. Thank you, team!

THANK YOU